Table des matières

Cours 05 — Le code d’un pipeline robuste 1
Objectifs 1
1. Ce qui sépare un script d’un pipeline 2
2. La configuration hors du code 3
3. Les secrets hors du dépôt 3
4. Une journalisation qui se lit, et qui se compte 4
5. Réessayer ce qui est passager 5
6. Tester une transformation 5
7. Une seule porte d’entrée 6
8. Empaqueter le pipeline 7
9. Un pipeline robuste, en pratique 8
À retenir 8
Exercices 9
Annexe A — Le détail des étapes du labo 9

Cours 05 — Le code d’un pipeline robuste

Théorie · Module 5/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → ingestion → lac → entrepôt → restitution, et cette fois **la chaîne elle-même**. Le cours ne va chercher aucune donnée nouvelle : il fait des scripts des Cours 02 à 04 un programme qu’on lance, relance, surveille et confie à un ordonnanceur, le service qui décide quand chaque traitement part et qui prévient quand il échoue (Airflow, Cours 06).

Objectifs

- Nommer ce qui sépare un script d’un pipeline.
- Sortir la configuration du code, et distinguer une valeur de configuration d’un secret.
- Journaliser pour deux lecteurs : celui qui cherche pourquoi, et celui qui compte combien.

© 2026 iAlexMG. Tous droits réservés.

- Distinguer une erreur passagère d'une erreur définitive, et borner les reprises.
- Écrire un test qui vérifie une promesse du pipeline, sans les données de production.
- Concevoir une porte d'entrée unique en ligne de commande, avec un code de sortie honnête.
- Dire ce qu'une image de conteneur fige, et ce qu'elle laisse dehors.

---

1. Ce qui sépare un script d'un pipeline

Le chargeur du Cours 03 fait bien son travail : il est idempotent, tient un journal des chargements et rend des comptes justes. Il n'est pourtant pas un pipeline, pour des raisons qui n'ont rien à voir avec ce qu'il calcule.

Ce qui manque	Ce que ça donne le jour où	La réponse du cours
Des valeurs écrites dans le code	le port de la base change, et il faut modifier un `.py` pour une valeur qui n'est pas du code	la configuration (section 2)
Un mot de passe quelque part	le dépôt part chez un collègue, ou sur GitHub, avec le mot de passe dedans	les secrets (section 3)
Des `print`	le pipeline a tourné cette nuit, personne ne regardait, et il ne reste rien	la journalisation (section 4)
Un appel réseau sans filet	l'API répond 503 pendant huit secondes, et toute l'exécution est perdue	les reprises (section 5)
Rien qui prouve que le code fait ce qu'il dit	un chiffre faux, sans la moindre erreur	les tests (section 6)
Une machine qui doit lancer le travail à votre place	il faut une commande, une seule, qui rende un code de sortie et tourne ailleurs que chez vous	la porte d'entrée et l'image (sections 7 et 8)

À retenir — robuste ne veut pas dire compliqué. Chacune de ces réponses tient en quelques dizaines de lignes. Ce qui coûte cher, c'est de les ajouter après, sur un code qui a grossi sans elles.

Le code de Boréal devient un paquet Python : un dossier de modules qui s'importe sous un seul nom, `pipeline`, comme vos scripts importent `duckdb`. Le langage lui-même, ses paquets et ses environnements appartiennent à la formation Python ; le terminal et les variables d'environnement, à la formation Linux. Ici, seulement ce qui rend un pipeline rejouable.

© 2026 iAlexMG. Tous droits réservés.

2. La configuration hors du code

- Une valeur de configuration change d'un environnement à l'autre sans que le programme change : un port, un chemin, une station de météo, un nombre d'essais. Une valeur du domaine fait partie du sens du programme : les 22 livraisons de Boréal et la forme de leurs fichiers restent dans le code.
- Le test : cette valeur serait-elle différente sur le poste d'un collègue, en préproduction, en production ? Si oui, elle sort ; sinon, la sortir ne ferait que disperser la lecture.

Forme	Ce qu'elle porte bien	Ce qu'elle porte mal
Un fichier de configuration (`config.toml`)	ce qui vaut pour tout le monde, structuré, commenté, versionné	ce qui change à chaque lancement, et tout secret
Une variable d'environnement	ce qui change d'une machine ou d'un conteneur à l'autre	une structure, une liste longue, une valeur qu'on veut relire dans six mois
Un argument de ligne de commande	ce qui change à chaque lancement : la date à traiter, l'étape à lancer	ce qui ne change jamais, et qu'on retaperait chaque fois

- Les trois se superposent, du plus général au plus précis : le fichier donne la valeur par défaut, l'environnement la remplace sur cette machine, l'argument la remplace pour ce lancement. Chez Boréal, `PIPELINE_ERP_PORT` gagne sur `[erp] port`.
- `config.toml` se lit par `tomllib`, dans la bibliothèque standard de Python depuis la version 3.11 : rien à installer, ce qui compte pour le chemin que rejoue le kit de départ d'Analytics Engineering, qui n'importe que DuckDB et la bibliothèque standard. YAML, plus répandu, revient au Cours 11 pour les contrats de données, où sa souplesse sert.

3. Les secrets hors du dépôt

Un secret n'est pas une configuration sensible : c'est autre chose. Une configuration se lit, se compare et se versionne, et son historique est utile. Un secret ne doit avoir aucun historique, parce qu'un dépôt Git n'oublie rien : un mot de passe retiré au commit suivant reste lisible dans le précédent, et l'en retirer vraiment demande de réécrire l'historique.

Règle	Chez Boréal	À plus grande échelle
Le secret vit hors du dépôt	`01_de/.env`, nommé dans le `.gitignore` du Cours 01	un gestionnaire de secrets, interrogé au démarrage
Le dépôt garde un modèle, jamais une valeur	`.env.exemple`, avec `ERP_MOT_DE_PASSE=remplacez-moi`	le même modèle, plus la procédure d'obtention
Le code ne connaît que le nom du secret	`secret("ERP_MOT_DE_PASSE")`	identique : seule la source de la valeur change

© 2026 iAlexMG. Tous droits réservés.

La troisième règle rend les deux autres tenables : tant que le code demande une valeur par son nom, l'endroit où elle dort peut changer sans que le code bouge (un `.env` sur votre poste, une variable d'environnement dans le conteneur, un gestionnaire de secrets en production).

Piège — le secret qui entre par la porte de derrière. Un mot de passe passé en argument de ligne de commande se retrouve dans l'historique du terminal et dans la liste des processus de la machine, où n'importe quel compte peut le lire. Un secret entre par un fichier ou par l'environnement, jamais par la ligne de commande.

4. Une journalisation qui se lit, et qui se compte

`print` écrit à l'écran, ce qui suffit tant que quelqu'un le regarde. Un pipeline qui tourne la nuit, dans un conteneur, appelé par un ordonnanceur, doit répondre à deux questions de nature différente.

Question	Qui la pose	Ce qu'il lui faut
Pourquoi cette exécution a-t-elle échoué ?	celui qui répare, une fois	un texte daté, avec le niveau de gravité et l'étape qui parle
Combien de fois, depuis quand, avec quel écart ?	celui qui surveille, tous les jours	des lignes de table, qu'on interroge en SQL

- Le module `logging` de Python répond à la première : il ajoute à chaque ligne son heure, son niveau et son origine, écrit au même moment à la console et dans un fichier, et laisse régler à partir de quel niveau une ligne s'écrit.

Niveau	Ce qu'on y met chez Boréal
`DEBUG`	le détail qui n'intéresse que pendant qu'on cherche : chaque page d'API, chaque table
`INFO`	le déroulement normal : l'étape commence, l'étape finit, avec ses comptes
`WARNING`	ce qui a été rattrapé : un essai raté, une reprise
`ERROR`	ce qui a arrêté le travail

- La table `pipeline.executions` répond à la seconde : une ligne par passage d'une étape, avec son identifiant, l'étape, le début, la fin, la durée, l'état, les lignes écrites et le message de l'erreur. Elle vit dans le schéma `pipeline` de l'entrepôt, à côté du journal des chargements du Cours 03.
- La ligne s'écrit avant que l'étape commence, à l'état « en cours », puis se complète à la fin. Si le processus est tué en plein travail, la ligne reste à « en cours », et une exécution qui n'a jamais fini se voit. Écrite seulement à la fin, elle disparaîtrait avec le processus.

À retenir — les deux traces ne se remplacent pas. Le fichier dit ce qui s'est passé, dans l'ordre, avec les détails. La table dit ce qui se passe d'habitude, et permet de comparer. Le Cours 06 s'en servira pour savoir ce qu'un DAG a vraiment fait, et le Cours 11 en partira pour l'observabilité.

© 2026 iAlexMG. Tous droits réservés.

Une trace qui vit dans la base qu'elle surveille a un point faible : l'étape travaille dans une transaction, et si elle échoue en plein milieu, tout ce qui passe par la même connexion échoue avec elle, y compris la ligne qui dirait pourquoi. Deux règles l'évitent :

- qui ouvre une transaction la referme, même quand ça rate : un `ROLLBACK` dans le chemin d'erreur, là où le `BEGIN` a été écrit ;
- la trace se ferme hors de toute transaction, sous son propre garde-fou : si elle n'arrive pas à s'écrire, c'est la cause réelle qui doit remonter, jamais l'erreur de journalisation.

C'est aussi pourquoi les outils d'observabilité vivent ailleurs : un système qui garde son état de santé dans la base qu'il surveille perd les deux d'un coup le jour où elle tombe. Ici, le compromis est assumé : la table sert à comparer des exécutions, pas à sonner l'alarme, et le fichier de journal est ailleurs.

5. Réessayer ce qui est passager

L'extraction météo du Cours 04 réessayait déjà. Le cours sort cette boucle du module de la météo et en fait un outil commun : la question se reposera pour toute tâche d'un DAG au Cours 06, et pour le courtier MQTT au Cours 09.

Nature de l'erreur	Exemples	Ce qu'on fait
Passagère	coupure du réseau, délai dépassé, `429` (trop de requêtes), `500` à `504` du serveur	réessayer, quelques fois, en attendant de plus en plus
Définitive	`401` (pas authentifié), `403` (pas autorisé), `404` (n'existe pas), une requête mal formée	abandonner tout de suite et le dire

- Réessayer une erreur définitive ajoute du délai et du bruit, sans rien changer. Abandonner sur une erreur passagère jette une exécution entière pour trois secondes de réseau.
- L'attente double à chaque échec : 2 secondes, puis 4, puis 8. Cette temporisation exponentielle sert autant la source que nous : un service en difficulté se rétablit plus vite si dix clients ne le rappellent pas chaque seconde.
- Le nombre d'essais est borné, et c'est une décision : quatre essais, trois attentes, quatorze secondes en tout, puis le pipeline échoue et le dit. Une boucle infinie de reprises transforme une panne visible en panne silencieuse qui tient la place jusqu'au matin.

Piège — réessayer ce qui n'est pas rejouable. Une reprise suppose que refaire l'appel est sans conséquence. C'est vrai d'une lecture ; ce n'est pas vrai d'une écriture qui ajoute. C'est pourquoi l'idempotence du Cours 03 vient avant les reprises : le chargeur se relance parce que son journal l'empêche de doubler quoi que ce soit.

6. Tester une transformation

© 2026 iAlexMG. Tous droits réservés.

Un pipeline se trompe rarement en plantant : il rend un chiffre faux, sans la moindre erreur. Les contrôles des Cours 02 à 04 l'attrapent sur les données réelles ; les tests l'attrapent avant, sur des données minuscules, en une fraction de seconde. Chacun vérifie une propriété, pas une ligne de code.

Le test	La promesse qu'il tient	Ce qu'il fabrique
Le journal refuse un extrait déjà chargé	relancer ne double rien ; un fichier corrigé remplace ses lignes ; un autre nom est une autre livraison	trois petits CSV de deux ou trois lignes
La conversion garde le compte des lignes	ce qui entre en CSV ressort en Parquet, sans perte ni doublon	deux CSV et un script de conversion réduit
La reprise s'arrête après le nombre d'essais prévu	quatre essais, trois attentes qui doublent, puis l'abandon ; un `404` ne passe qu'une fois	une fonction qui échoue toujours, et une horloge factice

- Le test fabrique ses données : il ne dépend ni de la génération de Boréal, ni d'un conteneur, ni du réseau, et tourne à chaque changement en moins d'une seconde. Un test qui demande deux minutes de mise en place ne sera pas lancé.
- Le test n'écrit rien dans le projet : le pipeline reçoit, par la variable `PIPELINE_PROJET`, un dossier temporaire que l'outil de test efface ensuite. C'est la même variable qui sert au conteneur (section 8) : ce qui rend un programme testable le rend souvent transportable.
- Ce qui ne se teste pas ainsi se nomme : la vraie API d'ECCC, le vrai pilote ODBC, les 138 Mo de Boréal. Ces chemins se vérifient en les exécutant, avec les contrôles des labos.

7. Une seule porte d'entrée

La règle de conception qui sert tout le reste du parcours : le pipeline s'appelle d'une seule façon.

```
python -m pipeline bronze raw corpus
```

Qui appelle	Comment	Quand
Vous, dans le terminal	`python -m pipeline raw`	pendant que vous travaillez
Le conteneur	la commande est le point d'entrée de l'image	section 8
Le DAG d'Airflow	une tâche par étape, qui lance la même commande	Cours 06
Le kit de départ d'Analytics Engineering	la même commande, sans Airflow	Cours 12

© 2026 iAlexMG. Tous droits réservés.

- Le code de sortie : 0 quand tout est passé, 1 quand une étape a échoué. C'est la seule chose qu'un ordonnanceur lit avec certitude ; un pipeline qui écrit « ERREUR » dans son journal et rend 0 sera compté comme réussi, et personne ne sera prévenu.
- L'ordre : `tout` ne lance que les étapes du chemin hors ligne. Un pipeline qu'on doit pouvoir rejouer sans réseau ne suppose ni l'API d'ECCC ni la base de l'ERP ; ces étapes restent appelables par leur nom, quand leurs conditions sont réunies.
- `python -m pipeline` cherche le paquet dans le dossier courant, puis dans ceux que nomme la variable `PYTHONPATH`. Le labo travaille depuis `Projet_Boreal` et pose `PYTHONPATH=01_de` ; l'image travaille dans le dossier du dépôt ; Airflow, au Cours 06, fera comme le labo.

8. Empaqueter le pipeline

Une image est un paquet figé qui contient un logiciel et tout ce dont il a besoin pour tourner. Un conteneur est une instance de cette image, en marche. On ne modifie pas un conteneur à la main : on le détruit et on le recrée depuis l'image, ce qui garantit que tout le monde fait tourner la même chose. C'est aussi pourquoi la base de l'ERP du Cours 04 se recharge par son script plutôt qu'à la main.

Analogie — l'image et le conteneur. L'image est le plan de l'ouvrage ; le conteneur est l'ouvrage bâti d'après ce plan. On en bâtit autant qu'on veut, on les démolit sans toucher au plan, et deux ouvrages bâtis d'après le même plan sont identiques.

Une image répond à « ça marche sur ma machine » : elle fige Python, DuckDB et le code, si bien que le même pipeline rend les mêmes chiffres chez vous, chez un collègue et sur le serveur qui l'ordonnancera.

Dans l'image	**Dehors, et pourquoi**
Python, dans une version nommée	les données : elles arrivent par un montage, et une image de 138 Mo de données ne se reconstruit pas
DuckDB, dans une version fixée (`duckdb==1.5.5`)	les secrets : ils entrent par l'environnement, au lancement
Le paquet `pipeline` et les scripts SQL	les tests : ils appartiennent à l'atelier, pas au pipeline livré
La configuration par défaut	les journaux : ils s'écrivent dans le dossier monté, et survivent au conteneur

- La version de DuckDB est fixée : sans elle, l'image reconstruite dans six mois prendrait la version du jour, et un moteur SQL qui change de version peut changer un type deviné ou un ordre de tri. Le Cours 01 a montré ce que la détection des types fait aux extraits de l'ERP.
- Le pipeline trouve le projet par `PIPELINE_PROJET`, qui vaut `/travail` dans l'image : la variable dont les tests se servent aussi. Un programme qui ne suppose pas où il tourne se déplace sans se réécrire.

Piège — le conteneur ne vit pas à la même heure que vous. Un conteneur Linux tourne en temps universel coordonné, UTC, tant qu'on ne lui dit pas autre chose. Le fichier de journal qu'il écrit porte donc parfois la date du lendemain : une raison de dater les journaux en UTC partout plutôt que de deviner.

© 2026 iAlexMG. Tous droits réservés.

9. Un pipeline robuste, en pratique

Un pipeline se juge sur ce qu'il fait quand personne ne regarde. L'ingénieur de données pose toujours les mêmes questions, et chacune a sa preuve.

Question	Ce qu'on vérifie	Chez Boréal (labo)
La configuration est-elle hors du code ?	une valeur remplacée sans toucher au code	le port 5433, puis 5434 sous `PIPELINE_ERP_PORT`, toujours un entier (2.3, 2.4)
Le pipeline rebâtit-il tout depuis les sources ?	un passage sur un entrepôt effacé	433 582 lignes de bronze, 1 572 fichiers et 513 857 lignes en `RAW`, 35 documents (6.4)
Se relance-t-il sans rien doubler ?	un passage sur un état déjà chargé	« à jour » sur les 22 livraisons (6.2, 8.6)
Un échec se voit-il ?	le code de sortie et la table des exécutions	1, et la ligne « échoué » de `erp` (6.5, 6.9)
Que s'est-il passé cette nuit ?	le fichier de journal	un fichier par jour, dans `plateforme\journaux` (6.6)
Le code tient-il ses promesses ?	des tests sur des données fabriquées	`3 passed`, en 0,3 s (7.3)
Tourne-t-il ailleurs que chez moi ?	l'image, sur le projet monté	« à jour » partout, dans le conteneur (8.6)
Le secret reste-t-il dehors ?	le `git status` d'avant le commit	aucun `.env` (9)

Ces preuves se font à la main ici. Le Cours 06 confie le pipeline à Airflow, et le Cours 11 en fait des contrats.

À retenir

- Un script devient un pipeline quand il survit à ce qui n'était pas prévu : une valeur qui change, un réseau qui tombe, une nuit sans témoin.
- La configuration sort du code parce qu'elle change d'un environnement à l'autre ; le secret sort du dépôt parce que Git n'oublie rien. Le code ne connaît que le nom du secret.
- Un journal de texte répond à « pourquoi », une table d'exécutions à « combien de fois ». La ligne s'écrit avant le travail, pas après.
- Une erreur passagère se réessaie avec une attente qui double, un nombre borné de fois ; une erreur définitive s'annonce tout de suite. On ne réessaie que ce qui est rejouable.
- Un test vérifie une promesse sur des données qu'il fabrique, en moins d'une seconde, sans toucher au projet.

© 2026 iAlexMG. Tous droits réservés.

- Une seule porte d'entrée, avec un code de sortie honnête : c'est ce que le conteneur lance, ce que le DAG appelle et ce que le kit rejoue.
- Une image fige les versions et laisse dehors les données, les secrets et les tests.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_05_DE.md`. Faites-les après le labo : plusieurs énoncés supposent que le paquet `pipeline` existe, que ses tests passent et que la table `pipeline.executions` a été remplie par quelques passages.

Annexe A — Le détail des étapes du labo

A.0 Avant de commencer : le dépôt tel que le Cours 04 l'a laissé

Notions : section 7, la porte d'entrée et `PYTHONPATH`.

- `set PYTHONPATH=01_de` ajoute `01_de` aux dossiers où Python cherche un paquet : `python -m pipeline` le trouvera depuis `Projet_Boreal`, où la fenêtre reste. `set` ne vaut que pour la fenêtre où on le tape, comme `conda activate`.
- Le dépôt porte les quatre scripts des Cours 03 et 04 et les quatre scripts SQL des Cours 01, 02 et 04 ; la `RAW` est chargée, 22 tables et 513 857 lignes.
- `pytest` est arrivé avec les paquets de la formation, à l'étape 1 du guide d'installation du parcours.

A.1 Étape 1 : créer le paquet du pipeline

Labo : `Labo_05_DE.md`, étape 1.

Notions : section 1, ce qui sépare un script d'un pipeline.

- 1.1. `pipeline` recevra tout le code du pipeline, `tests` ses tests. Les scripts des Cours 03 et 04 ne s'y déplacent pas : ils s'y réécrivent en modules (étapes 4 et 5), puis le dépôt les retire (5.5). Déplacer un fichier qu'on va réécrire n'apporte rien : Git ne garde pas les déplacements, il les devine au commit, à la ressemblance des contenus (étape 9).
- 1.2. `__init__.py` fait du dossier un paquet. Sa description sépare les modules de fond (`config`, `journal`, `reprise`, `entrepot`), qui servent toutes les étapes, des modules d'étape (`bronze`, `raw`, `corpus`, `erp`, `meteo`), qui ne connaissent que leur source.

A.2 Étape 2 : la configuration et le secret

Labo : `Labo_05_DE.md`, étape 2.

© 2026 iAlexMG. Tous droits réservés.

Notions : section 2, la configuration hors du code ; section 3, les secrets hors du dépôt.

- 2.1. `config.toml` range les valeurs par section : les chemins, relatifs à `Projet_Boreal`, le niveau du journal, les reprises, la base de l'ERP et l'API météo. Aucun mot de passe : le fichier va dans Git.
- 2.2. Ce que fait chaque morceau de `config.py` :
  - `DEPOT` et `projet()` : le dépôt est le dossier qui porte le paquet ; le projet est son parent, sauf si `PIPELINE_PROJET` en désigne un autre ;
  - `_surcharger` : une variable `PIPELINE_<SECTION>_<CLE>` remplace la valeur du fichier, convertie dans le type de celle-ci ;
  - `lru_cache` garde le résultat du premier appel : la configuration se lit une fois par exécution, et deux étapes d'un même passage ne travaillent jamais sur deux configurations différentes ;
  - `chemin()` rend un chemin par son nom ; `secret()` demande à l'environnement d'abord, au `.env` ensuite, ce qui laisse un conteneur travailler sans fichier de secrets.
- 2.3. `python -c` lance une ligne de Python : la configuration se lit depuis `Projet_Boreal`, grâce au `PYTHONPATH`.
- 2.4. L'environnement gagne sur le fichier, et le port reste un entier. `set PIPELINE_ERP_PORT=` retire la variable : oubliée, elle ferait chercher la base sur le mauvais port jusqu'à la fermeture de la fenêtre.
- Deux raisons, deux fichiers : `config.toml` est versionné, son historique raconte les décisions ; `.env` ne l'est jamais.

A.3 Étape 3 : le journal et l'entrepôt

Labo : `Labo_05_DE.md`, étape 3.

Notions : section 4, une journalisation qui se lit, et qui se compte.

- 3.1. Ce que fait chaque morceau de `journal.py` :
  - `demarrer` branche la console et un fichier par jour, `plateforme/journaux/pipeline_AAAA-MM-JJ.log`, au niveau que donne la configuration ;
  - `creer_table` crée `pipeline.executions` ; `Trace` est ce qu'une étape peut dire d'elle-même (les lignes écrites, un message) ;
  - `etape` est un bloc `with`, fabriqué par `contextmanager` : ce qui précède le `yield` se fait à l'entrée (la ligne « en cours »), ce qui suit à la sortie, même quand l'étape lève une erreur ;
  - le bloc `except` fait deux choses, dans cet ordre : annuler la transaction que l'étape aurait laissée ouverte (`ROLLBACK`), puis écrire la trace sous son propre `try` ; enfin il relance l'erreur, qui remonte à la porte d'entrée.

© 2026 iAlexMG. Tous droits réservés.

- 3.2. L'entrepôt n'est plus ouvert par chaque script, mais une seule fois, par la porte d'entrée, et passé aux étapes : un seul processus à la fois sur le fichier (décision D67 du parcours). `EntrepotOccupe` traduit le verrou de Windows en une phrase qui dit quoi faire ; `deja_charge` rend ce que le journal des chargements sait d'une table.

A.4 Étape 4 : les reprises en commun

Labo : `Labo_05_DE.md`, étape 4.

Notions : section 5, réessayer ce qui est passager.

- 4.1. `avec_reprises` lance l'action, la réessaie tant que `passagere` reconnaît l'erreur, attend `attente × 2^(essai - 1)` secondes entre deux essais, et lève `Abandon` au dernier. Sa façon d'attendre, `dormir`, se remplace dans les tests : la reprise s'y vérifie sans attendre quatorze secondes.
- 4.2. Trois changements dans le script de l'API, et rien d'autre : la boucle d'essais part dans `reprise.py`, la station, les dates et la taille de page viennent de la configuration, les `print` deviennent des lignes de journal. Le module garde ce qui lui est propre, savoir quelles erreurs HTTP valent un second essai (`passagere`), et une extraction incomplète lève au lieu de sortir.
- Ce qui se répète dans deux étapes n'appartient à aucune des deux : une reprise, une journalisation, une lecture de configuration sont l'affaire du pipeline ; l'étape ne garde que la connaissance de sa source.

A.5 Étape 5 : les étapes et la porte d'entrée

Labo : `Labo_05_DE.md`, étape 5.

Notions : section 7, une seule porte d'entrée ; section 2, la configuration.

- 5.1. `bronze.py` lance les scripts SQL du Cours 02, qui restent dans leurs fichiers, lançables seuls par `duckdb -f`. Il lit le contrôle que chaque script porte en dernière instruction : un écart entre la source et le bronze arrête le pipeline au lieu de défiler à l'écran, et un script qui ne finit pas par un contrôle à deux colonnes est refusé plutôt que cru. Les dossiers parents du `COPY` partitionné se créent avant (le refus du Cours 02, `Failed to create directory`).
- 5.2 à 5.4. Les trois modules réécrivent les scripts d'origine avec les mêmes retouches, comme `meteo.py` en 4.2 :

Ce qui part	**Ce qui le remplace**
`def main()`, le bloc `if __name__ == "__main__"`	`def executer(con)`, appelée par la porte d'entrée, qui se place dans `Projet_Boreal`
l'ouverture de l'entrepôt dans le script	la connexion reçue en argument : un seul processus, une seule connexion
les `print`	`LOG.info` et `LOG.debug`
les constantes en tête de fichier	`config()`, `chemin()` et `secret()`

© 2026 iAlexMG. Tous droits réservés.

- Chaque `executer` rend un entier, ce que l'étape a écrit : c'est ce que la porte d'entrée inscrit dans la colonne `lignes`. `raw.py` envoie le compte des fichiers au journal et rend celui des lignes ; une étape qui rendrait un couple ferait échouer l'écriture dans la table sur une erreur de conversion que rien ne relierait à sa cause. `a_charger` est sortie de `charger_une`, ce qui la rend testable.
- Ces modules sont écrits tels qu'ils sont au Cours 05, comme le code grandit dans un vrai projet : rien n'y prépare ce qu'un cours suivant demandera. Le Cours 06 y ajoutera la tranche de temps qu'une exécution traite, quand Airflow en aura besoin. Le `Solution/` porte l'état final du dépôt, ces ajouts compris.
- 5.3. `erp.py` importe `pyodbc` dans sa fonction, pas en tête du module : le pilote ne sert qu'à cette extraction, et importé en tête, il empêcherait tout le pipeline de démarrer sur un poste qui ne l'a pas.
- 5.5. `git rm` efface les fichiers du disque et range leur retrait dans l'index : le commit de l'étape 9 les montrera supprimés. Ils restent lisibles dans l'historique du dépôt, au commit du Cours 04.
- 5.6. Cinq modules d'étape, cinq fonctions `executer` : la porte d'entrée les cherche dès son import, et un module incomplet la fait mourir avant même d'ouvrir l'entrepôt.
- 5.7. Ce que fait chaque morceau de `__main__.py` :
  - un paquet qui porte un `__main__.py` se lance par `python -m <paquet>` ;
  - `ETAPES` est la seule liste des étapes : ajouter une étape au pipeline, plus tard, sera une ligne ici ;
  - `argparse`, de la bibliothèque standard, lit la ligne de commande, vérifie les arguments et écrit lui-même le message d'aide ; `resoudre` traduit `tout` en étapes du chemin hors ligne ;
  - `main` se place dans le projet, ouvre le journal, ouvre l'entrepôt une fois, lance chaque étape sous sa trace, rend 1 à la première erreur et ferme l'entrepôt quoi qu'il arrive.

A.6 Étape 6 : faire tourner le pipeline, et lire ses traces

Labo : `Labo_05_DE.md`, étape 6.

Notions : section 7, le code de sortie ; section 4, les deux traces.

- 6.1. `--etapes` liste sans rien lancer : la porte se lit avant de s'ouvrir.
- 6.2. Le bronze se refait toujours (`OVERWRITE`). La `RAW` reconnaît chaque fichier que le chargeur du Cours 03 avait chargé : même journal, mêmes chemins relatifs. Le corpus retrouve ses 35 empreintes.

© 2026 iAlexMG. Tous droits réservés.

- 6.3. On n'efface que ce que `tout` reconstruit à partir de `sources/`. Le reste de la plateforme ne vous appartient pas : le `docker-compose.yml`, le `Dockerfile.airflow` et le `.env` du mot de passe sont arrivés tout prêts au Cours 04, aucun cours ne les recrée, et Airflow en a besoin au Cours 06. Les bronzes `erp_base` et `meteo_api` restent aussi : `tout` ne les refait pas, puisqu'il leur faut la base et le réseau.
- 6.4. Un premier passage complet, les chiffres du Labo 03 : la même logique de chargement, sous une autre forme.
- 6.5. `PIPELINE_ERP_PORT=5439` envoie l'étape vers un port où rien n'écoute : l'étape échoue, la porte d'entrée l'inscrit dans la table et rend 1. `python -m pipeline erp` réussit, lui, quand la base du Cours 04 tourne sur le 5433.
- 6.6. Le fichier du jour reprend les lignes de la console. `--bavard` y ajoute le niveau `DEBUG`.
- 6.7 à 6.10. L'entrepôt s'attache en lecture seule : on regarde. La table garde une ligne par étape et par passage ; l'étape `erp` y est « échouée », sans lignes, avec le début du message. L'entrepôt se détache avant l'étape 8, qui écrit dedans.

A.7 Étape 7 : trois tests

Labo : `Labo_05_DE.md`, étape 7.

Notions : section 6, tester une transformation.

- 7.1. `pythonpath = .` met `01_de` dans les chemins d'importation des tests ; `testpaths` dit où les chercher.
- 7.2. Trois outils de `pytest` servent : `tmp_path` donne à chaque test un dossier vide à lui, effacé ensuite ; `monkeypatch` change une variable d'environnement ou le dossier courant le temps du test ; `cache_clear`, que `lru_cache` ajoute à la fonction qu'il décore, jette la configuration gardée, sans quoi le second test relirait celle du premier. Le troisième test ne dort jamais : il donne à `avec_reprises` une liste qui note les délais au lieu de les subir, et affirme qu'ils valent 2, 4 et 8 secondes.
- 7.3. `pytest 01_de` trouve le `pytest.ini` du dépôt, en fait sa racine, et lance les fonctions dont le nom commence par `test_`. Les trois tests n'ont touché ni `sources/`, ni la plateforme, ni le réseau. Un test doit échouer quand le code est faux : remplacer, dans `a_charger`, la condition `connus.get(f) != (sceau := empreinte(f))` par une condition toujours vraie doit faire tomber le premier ; s'il passe encore, c'est le test qu'il faut corriger.

A.8 Étape 8 : l'image

Labo : `Labo_05_DE.md`, étape 8.

Notions : section 8, empaqueter le pipeline.

- 8.1. Ce que fait chaque ligne du `Dockerfile` : `FROM` part d'une image de Python 3.11 réduite ; `pip install duckdb==1.5.5` fige le moteur ; `COPY` n'emporte que le paquet, les scripts SQL et la configuration ; `PIPELINE_PROJET=/travail` dit où le projet arrivera ; `PYTHONUNBUFFERED=1` fait sortir le journal au fil de l'eau. `ENTRYPOINT` fixe la commande que le conteneur lance toujours, `CMD` ses arguments par défaut, que la ligne de commande remplace : `docker run <image>` liste les étapes, `docker run <image> raw` lance `raw`.

© 2026 iAlexMG. Tous droits réservés.

- 8.2. Le Bloc-notes ajoute `.txt` à un nom qui n'a pas d'extension ; le point initial de `.dockerignore` lui suffit, pas le nom nu de `Dockerfile`.
- 8.3. Docker envoie le dossier au moteur avant de construire : ce que liste `.dockerignore` ne part pas, donc ne peut pas finir dans l'image par accident, à commencer par le `.env`.
- 8.4. `wsl` ouvre Ubuntu dans le dossier courant ; sous Ubuntu, le disque C de Windows se lit sous `/mnt/c`.
- 8.5. `docker build -t boreal-pipeline 01_de` construit l'image à partir du dépôt, qui porte le `Dockerfile`, et la nomme.
- 8.6. `-v "$PWD":/travail` monte le dossier courant, `Projet_Boreal`, dans le conteneur ; `--rm` efface le conteneur à la fin. Deux observations :
    - le lignage de la `RAW` est écrit en chemins relatifs, à barres obliques (`sources/erp/extraits/2023-01/VBAK.csv`) : c'est pourquoi le conteneur, sous Linux, reconnaît ce que Windows a chargé ; un chemin absolu, ou des barres inversées, auraient fait recharger les 1 572 fichiers ;
    - l'étape `raw` prend 6 à 7 s dans le conteneur contre moins d'une demi-seconde sous Windows, pour le même travail, 1 572 empreintes : les fichiers de Windows vus d'un conteneur Linux passent par une couche de traduction. C'est le coût annoncé par la décision D59 du parcours.
- Le conteneur date son journal en UTC : à partir de 20 h, heure de l'Est en été, il écrit dans le fichier du lendemain, pendant que Windows écrit dans celui du jour.

A.9 Étape 9 : versionner

Labo : `Labo_05_DE.md`, étape 9.

Notions : section 3, les secrets hors du dépôt.

- `git add` nomme ce qui entre : le paquet, les tests, la configuration, `pytest.ini` et les deux fichiers de l'image. Les `__pycache__` restent dehors par le `.gitignore` du Cours 01 ; le cache de `pytest` s'ignore de lui-même.
- Le `git status` d'avant le commit est le geste qui compte : on y vérifie que `.env` n'est pas dans la liste. Un mot de passe validé une fois reste dans l'historique.
- Git devine les déplacements à la ressemblance des contenus. Il reconnaît `pipeline/raw.py` dans `charger_raw.py`, semblable à 57 %, et le montre renommé ; les trois autres modules ont trop changé pour qu'il les reconnaisse, et leurs scripts apparaissent supprimés, les modules créés.
- L'image vit dans Docker, les données dans la plateforme : ni l'une ni les autres n'entrent dans le dépôt.

En entreprise, les mêmes gestes changent d'outil :

© 2026 iAlexMG. Tous droits réservés.

Geste du labo	En entreprise
La configuration hors du code (2)	un fichier par environnement (développement, préproduction, production), choisi au déploiement
Le secret demandé par son nom (2)	un coffre à secrets : AWS Secrets Manager, Azure Key Vault, les secrets de Databricks
Le journal et la table des exécutions (3, 6)	des journaux centralisés, et l'historique de l'orchestrateur ; Databricks garde celui de ses tâches
Les tests (7)	lancés à chaque changement par l'intégration continue, avant toute fusion
L'image (8)	construite par l'intégration continue, rangée dans un registre d'images, lancée par l'orchestrateur

© 2026 iAlexMG. Tous droits réservés.